

Project

Data Management for Big Data

Da Vinchie Lisa [SM3500574], Marsich Gaia [SM3500600]

A.A. 2022 - 2023, Data Science and Scientific Computing

Contents

1	General overview	2
1.1	Introduction	2
1.2	Creating the database	3
1.3	Populating the database	4
1.4	Some statistics	4
2	Query schemata	7
2.1	Query schema 1 - Export/import revenue value	7
2.1.1	Request	7
2.1.2	Implementation	7
2.2	Query schema 2 - Late delivery	11
2.2.1	Request	11
2.2.2	Implementation	12
3	Optimisation	14
3.1	Indexing on tables	14
3.1.1	Query schema 1 - Export/import revenue value	14
3.1.2	Query schema 2 - Late delivery	15
3.1.3	Optimising both query 1 and query 2	17
3.2	Materialisation of views	18
3.2.1	Query schema 1 - Export/import revenue value	18
3.2.2	Query schema 2 - Late delivery	20
3.3	Materialised view with indexes	20
3.4	Conclusions	21
A	Annex A - SQL definition of the tables	21

1 General overview

1.1 Introduction

The database we worked with was a collection of shipping information grouped in 8 tables (see Figure 1); the related documentation can be found here: https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v3.0.1.pdf. We were given the tables as .tbl files: customer.tbl, lineitem.tbl, nation.tbl, orders.tbl, part.tbl, partsupp.tbl, region.tbl, supplier.tbl.

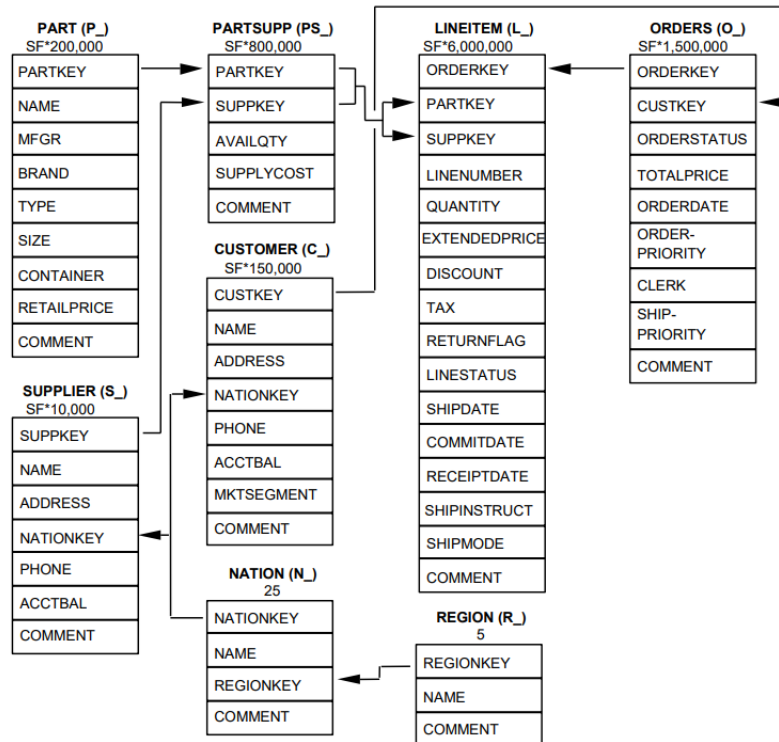


Figure 1: TPC-H schema. Here the relations between the tables are represented. The parentheses following each table name contain the prefix of the column names for that table. The arrows point in the direction of the one-to-many relationships between tables. The number/formula below each table name represents the cardinality (number of rows) of the table. Some are factored by SF, the Scale Factor, to obtain the chosen database size. The cardinality for the *linetitem* table is approximate

The purpose of this project was to practise database management, using in particular pgAdmin, by writing a set of queries and then studying an optimisation.

1.2 Creating the database

The first step was to prepare the database structure. For each table, the documentation gives indications on:

- Attribute types
- Primary keys
- Foreign keys
- Check constraints

To get a better idea of how the database is organised, we summarise here the information concerning primary keys and foreign keys (see Table 1):

Table name	Primary Key	Foreign Key
<i>customer</i>	c_custkey	c_nationkey (to n_nationkey)
<i>lineitem</i>	(l_orderkey, l_linenumber)	l_orderkey (to o_orderkey), l_partkey (to p_partkey), l_suppkey (to s_suppkey)
<i>nation</i>	n_nationkey	n_regionkey (to r_regionkey)
<i>orders</i>	o_orderkey	o_custkey (to c_custkey)
<i>part</i>	p_partkey	/
<i>partsupp</i>	(ps_partkey, ps_suppkey)	ps_partkey (to p_partkey), ps_suppkey (to s_suppkey)
<i>region</i>	r_regionkey	/
<i>supplier</i>	s_suppkey	s_nationkey (to n_nationkey)

Table 1: Primary keys and foreign keys of the tables

We do not report here the information on attribute types and on check constraints so as not to burden the discussion: in any case, the specific code we used is provided in **A Annex A - SQL definition of the tables**. The query is divided in three main parts:

- Creation of tables: performed with the command `CREATE TABLE`. We specified the primary keys, if possible the foreign keys (if the attribute they pointed to already existed), the other attributes and the related types
- Addition of missing foreign keys: due to the order we had chosen to create the tables, some links for the foreign keys were missing (one cannot create a link to something that does not exist yet)
- Addition of check constraints: we inserted all the check constraints described by the documentation

1.3 Populating the database

After setting up the structure of the database we had to populate the empty tables with the information contained in the `.tbl` files. At the very end of each line of the files there was a pipe symbol that prevented the action of populating the database. Therefore, we created a script in Python to remove this last character from each row of each file.

The new versions of the files were then used to populate the database. For each table we executed a query of this form:

```
1  -- Populate [table_name] table
2  COPY public.[table_name] FROM
3  '[path_to_file]'
4  DELIMITER '|' CSV;
```

where `[table_name]` and `[path]` had to be respectively substituted by the name of the table to be populated and the path of the corresponding `.tbl` file. Populating the 8 tables took in total 1 hour and 28 minutes.

1.4 Some statistics

In this section we collected some statistics related to the database we analysed. In Table 2 we reported an overview for each table in the database, writing down the amount of rows and the size of the table itself in terms of occupied memory. In the following tables (namely, Tables 3, 4, 5, 6, 7, 8, 9, 10) we collected the information about the number of distinct values for each attribute, as well as the minimum and the maximum values.

Table name	# rows	Size
<i>customer</i>	1500000	334 MB
<i>lineitem</i>	59986052	10074 MB
<i>nation</i>	25	24 kB
<i>orders</i>	15000000	2360 MB
<i>part</i>	2000000	363 MB
<i>partsupp</i>	8000000	1535 MB
<i>region</i>	5	24 kB
<i>supplier</i>	100000	20 MB

Table 2: Tables' statistics

Attribute	Number of distinct values	MinVal	MaxVal
c_acctbal	818834	-999.99	9999.99
c_nationkey	25	0	24
c_custkey	1500000	1	1500000
c_phone	1499963	(character)	
c_mktsegment	5	(character)	
c_comment	1496636	(text)	
c_name	1500000	(text)	
c_address	1500000	(text)	

Table 3: *customer* table

Attribute	Number of distinct values	MinVal	MaxVal
l_commitdate	2466	1992-01-31	1998-10-31
l_receiptdate	2555	1992-01-03	1998-12-31
l_linenumber	7	1	7
l_quantity	50	1	50
l_orderkey	15000000	1	60000000
l_extendedprice	1351462	900.91	104949.50
l_discount	11	0.00	0.10
l_tax	9	0.00	0.08
l_partkey	2000000	1	2000000
l_suppkey	100000	1	100000
l_shipdate	2526	1992-01-02	1998-12-01
l_comment	34378943	(text)	
l_returnflag	3	(character)	
l_linestatus	2	(character)	
l_shipinstruct	4	(character)	
l_shipmode	7	(character)	

Table 4: *lineitem* table

Attribute	Number of distinct values	MinVal	MaxVal
n_nationkey	25	0	24
n_regionkey	5	0	4
n_name	25	(character)	
n_comment	25	(text)	

Table 5: *nation* table

Attribute	Number of distinct values	MinVal	MaxVal
o_orderkey	15000000	1	60000000
o_custkey	999982	1	1499999
o_totalprice	11944103	838.05	558822.56
o_orderdate	2406	1992-01-01	1998-08-02
o_shippriority	1	0	0
o_orderpriority	5	(character)	
o_orderstatus	3	(character)	
o_clerk	10000	(character)	
o_comment	14097230	(text)	

Table 6: *orders* table

Attribute	Number of distinct values	MinVal	MaxVal
p_size	50	1	50
p_retailprice	31681	900.91	2098.99
p_partkey	2000000	1	2000000
p_brand	25	(character)	
p_type	150	(text)	
p_container	40	(character)	
p_comment	806046	(text)	
p_name	1999828	(text)	
p_mfgr	5	(character)	

Table 7: *part* table

Attribute	Number of distinct values	MinVal	MaxVal
ps_partkey	2000000	1	2000000
ps_suppkey	100000	1	100000
ps_availqty	9999	1	9999
ps_supplycost	99901	1.00	1000.00
ps_comment	7914164	(text)	

Table 8: *partsupp* table

Attribute	Number of distinct values	MinVal	MaxVal
r_regionkey	5	0	4
r_name	5	(character)	
r_comment	5	(text)	

Table 9: *region* table

Attribute	Number of distinct values	MinVal	MaxVal
s_nationkey	25	0	24
s_suppkey	100000	1	100000
s_acctbal	95588	-999.92	9999.93
s_comment	99983	(text)	
s_phone	100000	(character)	
s_name	100000	(character)	
s_address	100000	(text)	

Table 10: *supplier* table

2 Query schemata

2.1 Query schema 1 - Export/import revenue value

2.1.1 Request

The request was to aggregate the export/import of revenue between two different nations (E, I), where E is the nation of the *lineitem* supplier and I the nations of the *lineitem* customer (“export” means that the supplier is in the nation E and “import” means is in the nation I).

The revenue is obtained with this formula: $\text{l_extendedprice} * (1 - \text{l_discount})$.

The aggregations had to be performed with the following roll-up:

Month $\rightarrow$ Quarter $\rightarrow$ Year

Type

Nation $\rightarrow$ Region

The slicing was over *Type* and *Exporting nation*.

2.1.2 Implementation

Referring to the schema represented in Figure 1, the nation of the *lineitem* supplier and the nation of the *lineitem* customer are intrinsically given by, respectively, the attribute *l_suppkey* and the attribute *l_orderkey*, both foreign keys.

As a matter of fact, observe that *l_suppkey* is linked to *ps_suppkey* that is linked to *s_suppkey*, in the table *supplier*. Here, we will read the value of *s_nationkey*, that identifies the nation.

On the other hand, *l_orderkey* is linked to *o_orderkey*, and in the table *orders* we can find *o_custkey*. This last one is a foreign key to *c_custkey* in the table *customer*, that contains the attribute *c_nationkey*, identifying the nation.

These considerations give the basis to perform the JOINS of the tables.

A slicing over *Type* (i.e., *p_type*) and *Exporting nation* (i.e., *s_nationkey*) is asked. Since for the slicing we need to indicate specific values (e.g., *p.p_type* =

'ECONOMY ANODIZED NICKEL' and s.s_nationkey = '12') we decided to create a WHERE clause that should be uncommented after setting the values of p.p_type and s.s_nationkey.

The roll-up has been performed using a GROUP BY on the attributes of interest. The definition of the quarters have been specified in the SELECT clause, where we also exploited, using SUM, the formula provided to compute the revenue. The following is the complete code:

```

1  SELECT EXTRACT(YEAR FROM l_shipdate) AS year,
2      EXTRACT(MONTH FROM l_shipdate) AS month,
3      CASE
4          WHEN EXTRACT(MONTH FROM l_shipdate) BETWEEN 1 AND 3 THEN 'Q1'
5          WHEN EXTRACT(MONTH FROM l_shipdate) BETWEEN 4 AND 6 THEN 'Q2'
6          WHEN EXTRACT(MONTH FROM l_shipdate) BETWEEN 7 AND 9 THEN 'Q3'
7          ELSE 'Q4'
8      END AS quarter,
9      p.p_type AS part_type,
10     n_export.n_nationkey AS exporting_nation_key,
11     n_export.n_name AS exporting_nation,
12     r_export.r_regionkey AS exporting_region_key,
13     r_export.r_name AS exporting_region,
14     n_import.n_nationkey AS importing_nation_key,
15     n_import.n_name AS importing_nation,
16     r_import.r_regionkey AS importing_region_key,
17     r_import.r_name AS importing_region,
18     SUM(l_extendedprice * (1 - l_discount)) AS revenue
19 FROM lineitem l
20 JOIN part p ON l.l_partkey = p.p_partkey
21 JOIN supplier s ON l.l_suppkey = s.s_suppkey
22 JOIN nation n_export
23     ON s.s_nationkey = n_export.n_nationkey
24 JOIN region r_export
25     ON n_export.n_regionkey = r_export.r_regionkey
26 JOIN orders o ON l.l_orderkey = o.o_orderkey
27 JOIN customer c ON o.o_custkey = c.c_custkey
28 JOIN nation n_import
29     ON c.c_nationkey = n_import.n_nationkey
30 JOIN region r_import
31     ON n_import.n_regionkey = r_import.r_regionkey
32 -- REPLACE value1 AND value2 WITH WHAT YOU NEED, THEN UNCOMMENT THE FOLLOWING:
33 -- WHERE
34 --     p.p_type = 'value1'
35 --     AND s.s_nationkey = 'value2'
36 GROUP BY EXTRACT(YEAR FROM l_shipdate),
37     EXTRACT(MONTH FROM l_shipdate),

```



```

38     quarter,
39     p.p_type,
40     n_export.n_nationkey,
41     r_export.r_regionkey,
42     n_import.n_nationkey,
43     r_import.r_regionkey
44 ORDER BY year, quarter, month,
45     part_type, exporting_nation_key, exporting_region_key;
46

```

and, as it is, it took 22 minutes and 59 seconds to be executed. However, one could run the query several times and perhaps on different computers to make a statistical study, computing the mean and the standard deviation describing the time needed.

Remark that everything was performed on the basis of `n_nationkey` and `r_regionkey`, being them primary keys, but for the sake of clarity also `n_name` and `r_name` were reported in the final table. To get `r_name` we had to perform JOINS with the *region* table, otherwise not necessary (having already `n_regionkey` in *nation*).

To have an idea of the number of possibilities we have with the slicing we exploit the previous query and basically group by `p.p_type` and `s.s_nationkey`. We executed the following as it is:

```

1  SELECT
2      part_type,
3      exporting_nation_key,
4      COUNT(*) AS row_count
5  FROM (
6      SELECT
7          EXTRACT(YEAR FROM l_shipdate) AS year,
8          EXTRACT(MONTH FROM l_shipdate) AS month,
9          CASE
10             WHEN EXTRACT(MONTH FROM l_shipdate) BETWEEN 1 AND 3 THEN 'Q1'
11             WHEN EXTRACT(MONTH FROM l_shipdate) BETWEEN 4 AND 6 THEN 'Q2'
12             WHEN EXTRACT(MONTH FROM l_shipdate) BETWEEN 7 AND 9 THEN 'Q3'
13             ELSE 'Q4'
14          END AS quarter,
15      p.p_type AS part_type,
16      n_export.n_nationkey AS exporting_nation_key,
17      n_export.n_name AS exporting_nation,
18      r_export.r_regionkey AS exporting_region_key,
19      r_export.r_name AS exporting_region,
20      n_import.n_nationkey AS importing_nation_key,
21      n_import.n_name AS importing_nation,
22      r_import.r_regionkey AS importing_region_key,

```

```

23     r_import.r_name AS importing_region,
24     SUM(l_extendedprice * (1 - l_discount)) AS revenue
25 FROM
26     lineitem l
27 JOIN
28     part p ON l.l_partkey = p.p_partkey
29 JOIN
30     supplier s ON l.l_suppkey = s.s_suppkey
31 JOIN
32     nation n_export ON s.s_nationkey = n_export.n_nationkey
33 JOIN
34     region r_export ON n_export.n_regionkey = r_export.r_regionkey
35 JOIN
36     orders o ON l.l_orderkey = o.o_orderkey
37 JOIN
38     customer c ON o.o_custkey = c.c_custkey
39 JOIN
40     nation n_import ON c.c_nationkey = n_import.n_nationkey
41 JOIN
42     region r_import ON n_import.n_regionkey = r_import.r_regionkey
43 -- REPLACE value1 AND value2 WITH WHAT YOU NEED, THEN UNCOMMENT THE FOLLOWING:
44 -- WHERE
45     -- p.p_type = 'value1'
46     -- AND s.s_nationkey = 'value2'
47 GROUP BY
48     EXTRACT(YEAR FROM l_shipdate),
49     EXTRACT(MONTH FROM l_shipdate),
50     quarter,
51     p.p_type,
52     n_export.n_nationkey,
53     r_export.r_regionkey,
54     n_import.n_nationkey,
55     r_import.r_regionkey
56 ) AS subquery
57 GROUP BY
58     part_type,
59     exporting_nation_key;

```

and waited 11 minutes and 22 seconds. We get a table that summarises all the possible combinations we can set for the slicing. The number of rows of this result, equal to the number of possible combinations, is 3750. The number of rows of the table for each specific combination of `p.p_type` and `s.s_nationkey` is shown graphically in Figure 2.

We found out that the couple (`p.p_type`, `s.s_nationkey`) that would give the biggest table (in terms of amount of rows) is (LARGE BURNISHED COPPER, 3), with 2068 rows.

Let's consider this pair for the slicing, so the previously mentioned part to be

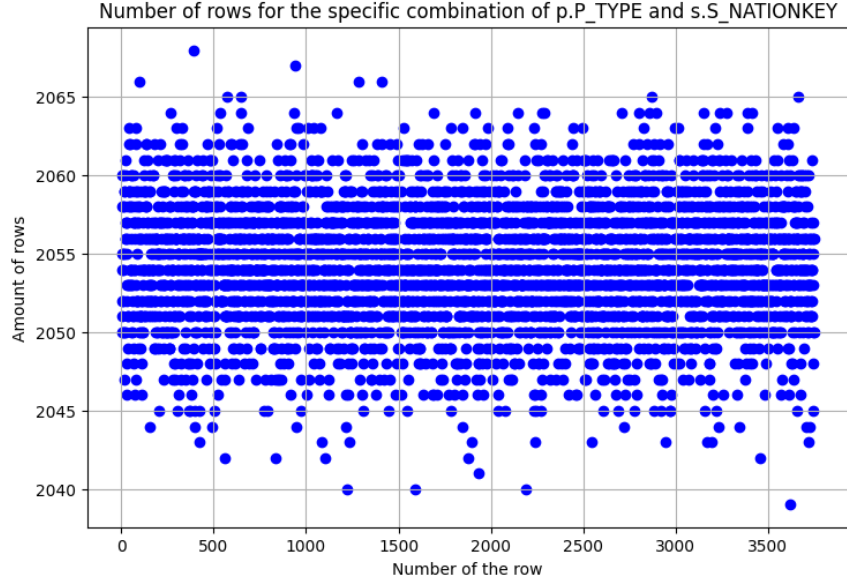


Figure 2: The graph reports, given a pair (p.p_type, s.s_nationkey), the amount of rows that the final table would have. On the x axis, each pair (p.p_type, s.s_nationkey) is identified by a number

decommented will become:

```

1 WHERE
2   p.p_type = 'LARGE BURNISHED COPPER'
3   AND s.s_nationkey = '3'

```

and the amount of time needed to execute the entire query was 1 minute and 13 seconds.

2.2 Query schema 2 - Late delivery

2.2.1 Request

The request was to display the number of orders where at least one item of the order had been received later than the committed date. The aggregations had to be performed with the following roll-up:

Month $\rightarrow$ Year

Nation $\rightarrow$ Region (customer)

Since no further information was provided, we assumed that the requested year

and month were the ones in which the order was shipped. Additionally, it was requested to perform a slicing over a specific *Month* and a specific *Type* of *lineitem*.

2.2.2 Implementation

Looking at the relational schema in Figure 1 we can see that the information that we need is scattered among the following columns:

- `o_orderkey`, that identifies the orders with a unique number
- `l_commitdate`, `l_receiptdate` and `l_shipdate`, that respectively contain the committed date, the receipt date and the date of shipping
- `n_nationkey`, that identifies a specific country
- `r_regionkey`, that identifies an area of the world
- `p_type`, that identifies the item type

In order to retrieve this information, we decided to join the tables *orders*, *customer*, *lineitem*, *nation*, *region* and *part* on their respective primary and foreign keys, with the condition that the date of receipt of the item was greater than the committed one. We called the temporary table `LateLineitemOrders` as it contains order key, shipping month, shipping year, nation key, region key and item type of all the items that arrived late.

From this table, it is very easy to retrieve the desired information, grouping by (year of `l_shipdate`, month of `l_shipdate`) and by (`n_regionkey`, `n_nationkey`), using the `ROLLUP` command; even though it is not required explicitly, it is mandatory to group by item type too as otherwise it is impossible to do the slicing.

Regarding the slicing, since it was not indicated which specific month and item type to use, we chose the combination that produced the table with the most rows: an approach similar to the one in section **2.1.2 Implementation** was used. The number of rows of the table for each specific combination of `p.p_type` and the month of `l.l_shipdate` is shown graphically in Figure 3.

The combination that would give the biggest table (in terms of amount of rows) is the one with shipping month 7 and item type `ECONOMY ANODIZED COPPER`.

The full query is reported below; it took 2 minutes and 37 seconds to complete.

```

1 WITH lateLineitemOrders AS (
2     SELECT o.o_orderkey AS OrderKey,
3           l.l_shipdate AS ShipDate,
4           EXTRACT(YEAR FROM l.l_shipdate) AS ShipYear,
5           EXTRACT(MONTH FROM l.l_shipdate) AS ShipMonth,
6           c.c_nationkey AS NationKey,
7           n.n_regionkey AS RegionKey,
8           p.p_type AS ItemType

```

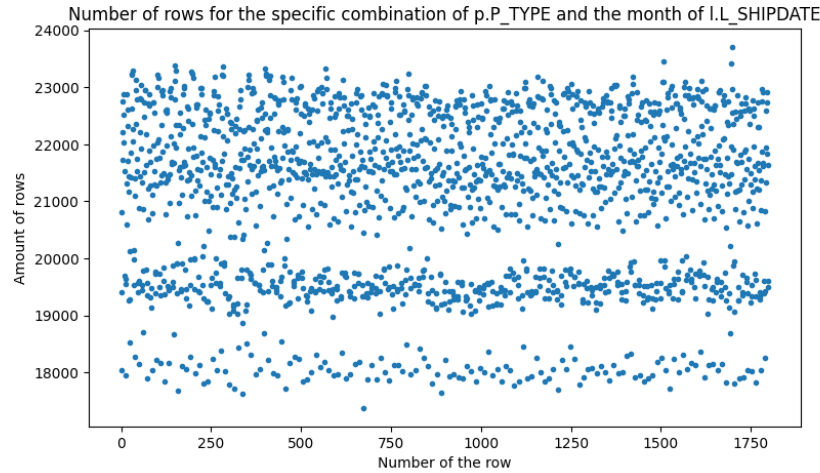


Figure 3: The graph reports, given a pair (p.p.type, month of l.l.shipdate), the amount of rows that the final table would have. On the x axis, each pair (p.p.type, month of l.l.shipdate) is identified by a number

```

9      FROM Orders o
10         INNER JOIN customer c ON o.o_custkey = c.c_custkey
11         INNER JOIN lineitem l ON o.o_orderkey = l.l_orderkey
12         INNER JOIN nation n ON c.c_nationkey = n.n_nationkey
13         INNER JOIN region r ON n.n_regionkey = r.r_regionkey
14         INNER JOIN part p ON l.l_partkey = p.p_partkey
15     WHERE l.l_receiptdate > l.l_commitdate
16 )
17 SELECT COUNT(DISTINCT llo.OrderKey) AS N_LateOrders,
18        llo.ShipYear AS ShipYear,
19        llo.ShipMonth AS ShipMonth,
20        llo.NationKey AS CustomerNationKey,
21        llo.RegionKey AS CustomerRegionKey,
22        llo.ItemType AS ItemType
23 FROM lateLineitemOrders AS llo
24 WHERE llo.ShipYear IS NOT NULL
25        AND llo.ShipMonth IS NOT NULL
26        AND llo.NationKey IS NOT NULL
27        AND llo.RegionKey IS NOT NULL
28        AND llo.ItemType IS NOT NULL
29        AND llo.ShipMonth = 7
30        AND ItemType = 'ECONOMY ANODIZED COPPER'
31 GROUP BY ROLLUP(llo.ShipYear, llo.ShipMonth),
32           ROLLUP(llo.RegionKey, llo.NationKey),

```

3 Optimisation

The request was to perform an optimisation that considers:

- Indexing on tables (mandatory)
- Materialisation of views (mandatory)
- Indexing on materialised views (optional)
- Horizontal partitioning (optional)

The final optimisation must fulfil the space constraint that the amount of space for indexes and materialised views must be less than 1.5 times the size of the database. The space occupied by the database is, rounded to MB, 14686 MB (we just had to sum the values of the tables' sizes written in Table 2), so the constraint is approximately $14686 \text{ MB} \times 1.5 = 22029 \text{ MB}$.

We consider the query schema 1 with the slicing on (`p.p_type`, `s.s_nationkey`) having the values (LARGE BURNISHED COPPER, 3) and the query schema 2 with the slicing on (`p.p_type`, month of `l.l_shipdate`) having the values (7, ECONOMY ANODIZED COPPER).

3.1 Indexing on tables

3.1.1 Query schema 1 - Export/import revenue value

First of all let's observe what happens if we add indexes. We firstly created indexes on all the attributes that are useful in executing the query schema 1:

```

1 CREATE INDEX idx_part_type ON part (p_type);
2 CREATE INDEX idx_supplier_nationkey ON supplier (s_nationkey);
3
4 CREATE INDEX idx_lineitem_partkey ON lineitem (l_partkey);
5 CREATE INDEX idx_part_partkey ON part (p_partkey);
6 CREATE INDEX idx_lineitem_suppkey ON lineitem (l_suppkey);
7 CREATE INDEX idx_supplier_suppkey ON supplier (s_suppkey);
8 CREATE INDEX idx_nation_nationkey ON nation (n_nationkey);
9 CREATE INDEX idx_nation_regionkey ON nation (n_regionkey);
10 CREATE INDEX idx_region_regionkey ON region (r_regionkey);
11 CREATE INDEX idx_lineitem_orderkey ON lineitem (l_orderkey);
12 CREATE INDEX idx_orders_orderkey ON orders (o_orderkey);
13 CREATE INDEX idx_orders_custkey ON orders (o_custkey);

```

```
14 CREATE INDEX idx_customer_custkey ON customer (c_custkey);
15 CREATE INDEX idx_customer_nationkey ON customer (c_nationkey);
```

The creation of indexes took 3 minutes and 58 seconds.

With these indexes, the query schema 1 took 24 seconds to be executed. To understand which indexes were actually used in performing that query, we had a look at the query execution plan using `EXPLAIN ANALYZE`, finding out that the relevant indexes were:

- `idx_part_type`
- `idx_lineitem_partkey`
- `idx_supplier_nationkey`
- `idx_orders_orderkey`
- `idx_customer_custkey`

We deleted all the indexes; the creation of the 5 relevant indexes took 1 minute and 50 seconds and they occupy 2654 MB of disk space.

The execution of the query schema 1 now takes 30 seconds, that is less than half of the not optimised query described in **2.1 Query schema 1 - Export/import revenue value**.

3.1.2 Query schema 2 - Late delivery

Since creating indexes requires time, we decided to create indexes only for the columns that are used for joining tables, for filtering or by the `SELECT` clause. To create the indexes, we tried three different strategies:

1. Using hash indexing instead of the default b-tree indexing whenever the column was used for joining, as hash indexing should be better whenever we need to handle equality comparisons
2. Using the default b-tree indexing and creating one index for each column
3. Using the default b-tree indexing and creating one index for each table, for multiple columns

The parameters of the three strategies are reported in Table 11; from this table, we can see how the second and the third strategies seem to be the most convenient, in terms of both occupied space and creation time.

The method that one may decide to use depends on how much the database is likely to change and on the number of queries that will be executed. From now on we will perform all the tests using the second option, that has the best query time and does not ask for too much memory.

The followings are the codes that we used:

Strategy	Indexes creation time [mm:ss]	Query time [mm:ss]	Occupied space [MB]
1	15:11	00:42	9876
2	06:54	00:30	4993
3	2:25	1:40	4442

Table 11: Parameters of the three strategies to create indexes

```

1  -- Strategy 1
2  CREATE INDEX idx_orders_orderkey ON orders USING HASH (o_orderkey);
3  CREATE INDEX idx_orders_custkey ON orders USING HASH (o_custkey);
4
5  CREATE INDEX idx_customer_custkey ON customer USING HASH (c_custkey);
6  CREATE INDEX idx_customer_nationkey ON customer USING HASH (c_nationkey);
7
8  CREATE INDEX idx_lineitem_orderkey ON lineitem USING HASH (l_orderkey);
9  CREATE INDEX idx_lineitem_partkey ON lineitem USING HASH (l_partkey);
10 CREATE INDEX idx_lineitem_supkey ON lineitem USING HASH (l_supkey);
11 CREATE INDEX idx_lineitem_btree ON lineitem USING BTREE (l_receiptdate,
    ↪ l_commitdate, l_shipdate);
12
13 CREATE INDEX idx_nation_nationkey ON nation USING HASH (n_nationkey);
14 CREATE INDEX idx_nation_regionkey ON nation USING HASH (n_regionkey);
15
16 CREATE INDEX idx_region_regionkey ON region USING HASH (r_regionkey);
17
18 CREATE INDEX idx_part_partkey ON part USING HASH (p_partkey);
19 CREATE INDEX idx_part_type ON part USING BTREE (p_type);

```

```

1  -- Strategy 2
2  CREATE INDEX idx_orders_orderkey ON orders (o_orderkey);
3  CREATE INDEX idx_orders_custkey ON orders (o_custkey);
4
5  CREATE INDEX idx_customer_custkey ON customer (c_custkey);
6  CREATE INDEX idx_customer_nationkey ON customer (c_nationkey);
7
8  CREATE INDEX idx_lineitem_orderkey ON lineitem (l_orderkey);
9  CREATE INDEX idx_lineitem_partkey ON lineitem (l_partkey);
10 CREATE INDEX idx_lineitem_supkey ON lineitem (l_supkey);
11 CREATE INDEX idx_lineitem_receiptdate ON lineitem (l_receiptdate);
12 CREATE INDEX idx_lineitem_commitdate ON lineitem (l_commitdate);
13 CREATE INDEX idx_lineitem_shipdate ON lineitem (l_shipdate);
14

```



```

15 CREATE INDEX idx_nation_nationkey ON nation (n_nationkey);
16 CREATE INDEX idx_nation_regionkey ON nation (n_regionkey);
17
18 CREATE INDEX idx_region_regionkey ON region (r_regionkey);
19
20 CREATE INDEX idx_part_partkey ON part (p_partkey);
21 CREATE INDEX idx_part_type ON part (p_type);

```

```

1 -- Strategy 3
2 CREATE INDEX idx_orders ON orders (o_orderkey, o_custkey);
3 CREATE INDEX idx_customer ON customer (c_custkey, c_nationkey);
4 CREATE INDEX idx_lineitem ON lineitem (l_orderkey, l_partkey, l_suppkey,
  ↳ l_receiptdate, l_commitdate, l_shipdate);
5 CREATE INDEX idx_nation ON nation (n_nationkey, n_regionkey);
6 CREATE INDEX idx_region ON region (r_regionkey);
7 CREATE INDEX idx_part ON part (p_partkey, p_type);

```

As we did in the previous section, we decided to use `EXPLAIN ANALYZE` to understand which indexes were in fact used during the query. We learned that only four indexes were actually used:

- `idx_part_type` on the `p_type` attribute of the *part* table
- `idx_lineitem_partkey` on the `l_partkey` attribute of the *lineitem* table
- `idx_orders_orderkey` on the `o_orderkey` attribute of the *orders* table
- `idx_customer_custkey` on the `c_custkey` attribute of the *customer* table

The creation of those indexes took only 1 minute and 50 seconds and they only occupy 2653 MB of disk space, that is much less than the version with all the indexes, while the query still takes 30 seconds to execute.

3.1.3 Optimising both query 1 and query 2

As we can see from the codes above, the two queries have all the indexes in common except for `idx_supplier_nationkey`, that is useful only for the first query. As we saw in **3.1.1 Query schema 1 - Export/import revenue value**, creating all the common indexes plus `idx_supplier_nationkey` requires 1 minute and 50 seconds with an occupied memory of 2654 MB. Compared to the results we got at the end of **3.1.2 Query schema 2 - Late delivery** (i.e., the part in which we worked with just the common indexes), we observe that creating `idx_supplier_nationkey` does not affect significantly neither the creation time nor the needed memory.

Therefore, as the best optimisation using indexes we would choose to use all the common indexes together with `idx_supplier_nationkey`.

3.2 Materialisation of views

We compared the query schemata 1 and 2 finding out the common tables and attributes that were involved. We therefore wrote down a query to get a materialised view that can be exploited in both cases:

```
1 CREATE MATERIALIZED VIEW materialised_view AS
2 SELECT
3     l.l_shipdate,
4     l.l_extendedprice,
5     l.l_discount,
6     l.l_suppkey,
7     l.l_commitdate,
8     l.l_receiptdate,
9     o.o_orderkey AS OrderKey,
10    p.p_type AS part_type,
11    n_import.n_nationkey AS importing_nation_key,
12    n_import.n_name AS importing_nation,
13    r_import.r_regionkey AS importing_region_key,
14    r_import.r_name AS importing_region
15 FROM
16     lineitem l
17 JOIN
18     part p ON l.l_partkey = p.p_partkey
19 JOIN
20     orders o ON o.o_orderkey = l.l_orderkey
21 JOIN
22     customer c ON o.o_custkey = c.c_custkey
23 JOIN
24     nation n_import ON c.c_nationkey = n_import.n_nationkey
25 JOIN
26     region r_import ON n_import.n_regionkey = r_import.r_regionkey
```

The query was executed in 9 minutes and 6 seconds; the materialised view occupies 8840 MB on disk. So, the space occupied by the materialised view is less than 1.5 times the size of the database: this constraint then is satisfied.

3.2.1 Query schema 1 - Export/import revenue value

To get the same result as the initial version of the query schema 1, the information given by `materialised_view` had to be supplemented with the data related to the suppliers, saved in the table `supplier`. We therefore had to perform a couple of JOINS.

Similarly to the original version, a `GROUP BY` clause was used to perform the roll-up while the relevant rows and columns were identified, respectively, with a `WHERE` and a `SELECT`.

The query below took 42 seconds to complete: we observed a great improve-

ment with respect to the query not optimised (the new one takes 58% of the time of the old one). However, on the whole the optimisation performed in **3.1.1 Query schema 1 - Export/import revenue value** appears to be slightly better since both the pre-processing of the materialised view and the execution of the query take more time.

Here is the code we used:

```

1  SELECT
2      EXTRACT(YEAR FROM mv.l_shipdate) AS year,
3      EXTRACT(MONTH FROM mv.l_shipdate) AS month,
4      CASE
5          WHEN EXTRACT(MONTH FROM mv.l_shipdate) BETWEEN 1 AND 3 THEN 'Q1'
6          WHEN EXTRACT(MONTH FROM mv.l_shipdate) BETWEEN 4 AND 6 THEN 'Q2'
7          WHEN EXTRACT(MONTH FROM mv.l_shipdate) BETWEEN 7 AND 9 THEN 'Q3'
8          ELSE 'Q4'
9      END AS quarter,
10     mv.part_type AS part_type,
11     n_export.n_nationkey AS exporting_nation_key,
12     n_export.n_name AS exporting_nation,
13     r_export.r_regionkey AS exporting_region_key,
14     r_export.r_name AS exporting_region,
15     mv.importing_nation_key,
16     mv.importing_nation,
17     mv.importing_region_key,
18     mv.importing_region,
19     SUM(l_extendedprice * (1 - l_discount)) AS revenue
20 FROM materialised_view mv
21 JOIN supplier s ON mv.l_suppkey = s.s_suppkey
22 JOIN nation n_export
23     ON s.s_nationkey = n_export.n_nationkey
24 JOIN region r_export
25     ON n_export.n_regionkey = r_export.r_regionkey
26 -- REPLACE value1 AND value2 WITH WHAT YOU NEED, THEN DECOMMENT THE FOLLOWING:
27 WHERE mv.part_type = 'LARGE BURNISHED COPPER'
28     AND s.s_nationkey = '3'
29 GROUP BY EXTRACT(YEAR FROM mv.l_shipdate),
30     EXTRACT(MONTH FROM mv.l_shipdate),
31     quarter,
32     mv.part_type,
33     n_export.n_nationkey,
34     r_export.r_regionkey,
35     mv.importing_nation_key,
36     mv.importing_nation,
37     mv.importing_region_key,
38     mv.importing_region
39 ORDER BY year, quarter, month,
40     part_type,

```

```

41      exporting_nation_key, exporting_region_key;

```

3.2.2 Query schema 2 - Late delivery

Using the same materialised view of the previous section, we rewrote the query for the schema 2 as below:

```

1  SELECT
2  COUNT(DISTINCT m.OrderKey) AS N_LateOrders,
3  EXTRACT(YEAR FROM m.ShipDate) AS ShipYear,
4  EXTRACT(MONTH FROM m.ShipDate) AS ShipMonth,
5  m.importing_nation_key AS NationKey,
6  m.importing_region_key AS RegionKey,
7  m.part_type AS ItemType
8  FROM materialised_view as m
9  WHERE m.ReceiptDate > m.CommitDate
10 AND EXTRACT(MONTH FROM m.ShipDate) = 7
11 AND m.part_type = 'ECONOMY ANODIZED COPPER'
12 GROUP BY
13     ROLLUP(EXTRACT(YEAR FROM m.ShipDate),
14            EXTRACT(MONTH FROM m.ShipDate)),
15     ROLLUP(m.importing_region_key, m.importing_nation_key),
16     m.part_type

```

This query requires 41 seconds to be executed, that is slightly worse than the indexing in **3.1.2 Query schema 2 - Late delivery**.

3.3 Materialised view with indexes

We decided to try to add indexes to the materialised view in order to further improve the performance of the queries. The indexes were generated as shown below:

```

1  CREATE INDEX idx_shipdate ON materialised_view (ShipDate);
2  CREATE INDEX idx_extendedprice ON materialised_view (ExtendedPrice);
3  CREATE INDEX idx_discount ON materialised_view (Discount);
4  CREATE INDEX idx_supplierkey ON materialised_view (SupplierKey);
5  CREATE INDEX idx_commitdate ON materialised_view (CommitDate);
6  CREATE INDEX idx_receiptdate ON materialised_view (ReceiptDate);
7  CREATE INDEX idx_orderkey ON materialised_view (OrderKey);
8  CREATE INDEX idx_part_type ON materialised_view (part_type);
9  CREATE INDEX idx_importing_nation_key
10     ON materialised_view (importing_nation_key);
11 CREATE INDEX idx_importing_nation ON materialised_view (importing_nation);

```

```

12 CREATE INDEX idx_importing_region_key
13     ON materialised_view (importing_region_key);
14 CREATE INDEX idx_importing_region ON materialised_view (importing_region);

```

The indexes took 20 minutes and 19 seconds to be created and occupy 7366 MB of disk space; with them, the queries for the schemas 1 and 2 took, respectively, 30 and 28 seconds to complete, roughly 10 seconds less than without indexes. By using `EXPLAIN ANALYZE` again, we discovered that just one index was actually used: the one on the `part_type` column. Creating this single index took 4 minutes and 57 seconds and the required space is 1857 MB; summing this to the space needed for the materialised view, we obtain that the total required space is 10697 MB, that is much less than 1.5 times the size of the database. So, the use of indexes on the materialised view improves query times, but we think that this is hardly convenient: the time required to create the materialised view and the indexes is more or less 14 minutes, that is much more than the 1 minute and 50 seconds needed to create the indexes of **3.1.3 Optimising both query 1 and query 2**.

3.4 Conclusions

In conclusion, we think that the best solution is to use indexing on tables. As we can see from Table 12, this method significantly reduces query time while requiring little time for pre-processing, and it also occupies the lowest disk space of all the optimisation methods.

Optimisation method	Preprocessing time [mm:ss]	Query time [mm:ss]	Disk space [MB]
None	00:00	3:50	0
Indexing on tables	01:50	01:00	2654
Materialised View	09:06	01:23	8840
Materialised view + indexes	14:03	00:58	10697

Table 12: Summary of all the optimisation methods used. The query time column indicates the sum of the times needed to complete the queries 1 and 2

A Annex A - SQL definition of the tables

The structure of the database was built following the instructions mentioned in this file: https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v3.0.1.pdf, in particular pp. 13 - 19.

Here is the code used to prepare the structure of the database before populating it:

```
1  -- PART 1: CREATE TABLES
2
3  CREATE TABLE IF NOT EXISTS public.customer
4  (
5      c_custkey INT PRIMARY KEY, -- Identifier SF*150,000 are populated
6      c_name VARCHAR(25), -- variable text, size 25
7      c_address VARCHAR(40), -- variable text, size 40
8      c_nationkey INT, -- Identifier Foreign Key to n_nationkey
9      c_phone CHAR(15), -- fixed text, size 15
10     c_acctbal DECIMAL,
11     c_mktsegment CHAR(10), -- fixed text, size 10
12     c_comment VARCHAR(117) -- variable text, size 117
13
14     -- FOREIGN KEY (c_nationkey) REFERENCES nation(n_nationkey)
15     -- ON DELETE CASCADE ON UPDATE CASCADE
16 )
17 TABLESPACE pg_default;
18
19 ALTER TABLE IF EXISTS public.customer
20     OWNER TO postgres;
21
22
23
24 CREATE TABLE IF NOT EXISTS public.lineitem
25 (
26     l_orderkey INT,
27     l_partkey INT,
28     l_suppkey INT,
29     l_linenum INT,
30     l_quantity DECIMAL,
31     l_extendedprice DECIMAL,
32     l_discount DECIMAL,
33     l_tax DECIMAL,
34     l_returnflag CHAR(1), -- fixed text, size 1
35     l_linestatus CHAR(1), -- fixed text, size 1
36     l_shipdate DATE,
37     l_commitdate DATE,
38     l_receiptdate DATE,
39     l_shipinstruct CHAR(25), -- fixed text, size 25
40     l_shipmode CHAR(10), -- fixed text, size 10
41     l_comment VARCHAR(44), -- variable text size 44
42     PRIMARY KEY (l_orderkey, l_linenum)
43     -- FOREIGN KEY (l_orderkey) REFERENCES orders(o_orderkey),
44     -- FOREIGN KEY (l_partkey, l_suppkey) REFERENCES partsupp(ps_partkey,
45     ↪ ps_suppkey)
```

```

45 )
46 TABLESPACE pg_default;
47
48 ALTER TABLE IF EXISTS public.lineitem
49     OWNER TO postgres;
50
51
52
53 CREATE TABLE IF NOT EXISTS public.region
54 (
55     r_regionkey INT PRIMARY KEY, -- identifier 5 regions are populated
56     r_name CHAR(25), -- fixed text, size 25
57     r_comment VARCHAR(152) -- variable text, size 152
58 )
59 TABLESPACE pg_default;
60
61 ALTER TABLE IF EXISTS public.region
62     OWNER TO postgres;
63
64
65
66 CREATE TABLE IF NOT EXISTS public.nation
67 (
68     n_nationkey INT PRIMARY KEY, -- identifier 25 nations are populated
69     n_name CHAR(25), -- fixed text, size 25
70     n_regionkey INT, -- Foreign Key to r_regionkey
71     n_comment VARCHAR(152) -- variable text, size 152
72
73     -- FOREIGN KEY (n_regionkey) REFERENCES region(r_regionkey)
74     -- ON DELETE CASCADE ON UPDATE CASCADE
75 )
76 TABLESPACE pg_default;
77
78 ALTER TABLE IF EXISTS public.nation
79     OWNER TO postgres;
80
81
82
83 CREATE TABLE IF NOT EXISTS public.part
84 (
85     p_partkey INT PRIMARY KEY, -- identifier SF*200,000 are populated
86     p_name VARCHAR(55), -- variable text, size 55
87     p_mfgr CHAR(25), -- fixed text, size 25
88     p_brand CHAR(10), -- fixed text, size 10
89     p_type VARCHAR(25), -- variable text, size 25
90     p_size INT,
91     p_container CHAR(10), -- fixed text, size 10

```

```

92     p_retailprice DECIMAL,
93     p_comment VARCHAR(23) -- variable text, size 23
94 )
95 TABLESPACE pg_default;
96
97 ALTER TABLE IF EXISTS public.part
98     OWNER TO postgres;
99
100
101
102 CREATE TABLE IF NOT EXISTS public.supplier
103 (
104     s_suppkey INT PRIMARY KEY, -- identifier SF*10,000 are populated
105     s_name CHAR(25), -- fixed text, size 25
106     s_address VARCHAR(40), -- variable text, size 40
107     s_nationkey INT, -- Identifier Foreign Key to n_nationkey
108     s_phone CHAR(15), -- fixed text, size 15
109     s_acctbal DECIMAL,
110     s_comment VARCHAR(101), -- variable text, size 101
111
112     FOREIGN KEY (s_nationkey) REFERENCES nation(n_nationkey)
113     ON DELETE CASCADE ON UPDATE CASCADE
114 )
115 TABLESPACE pg_default;
116
117 ALTER TABLE IF EXISTS public.supplier
118     OWNER TO postgres;
119
120
121
122 CREATE TABLE IF NOT EXISTS public.partsupp
123 (
124     ps_partkey INT,
125     ps_suppkey INT,
126     ps_availqty INT,
127     ps_supplycost DECIMAL,
128     ps_comment VARCHAR(199),
129     PRIMARY KEY (ps_partkey, ps_suppkey),
130     FOREIGN KEY (ps_partkey) REFERENCES part(p_partkey),
131     FOREIGN KEY (ps_suppkey) REFERENCES supplier(s_suppkey)
132 )
133 TABLESPACE pg_default;
134
135 ALTER TABLE IF EXISTS public.partsupp
136     OWNER TO postgres;
137
138

```



```

139
140 CREATE TABLE IF NOT EXISTS public.orders
141 (
142     o_orderkey INT PRIMARY KEY, -- Identifier SF*1,500,000 are sparsely
        ↳ populated
143     o_custkey INT, -- Foreign Key to c_custkey
144     o_orderstatus CHAR(1), -- fixed text, size 1
145     o_totalprice DECIMAL,
146     o_orderdate DATE,
147     o_orderpriority CHAR(15), -- fixed text, size 15
148     o_clerk CHAR(15), -- fixed text, size 15
149     o_shippriority INT,
150     o_comment VARCHAR(79), -- variable text, size 79
151
152     FOREIGN KEY (o_custkey) REFERENCES customer(c_custkey)
153     ON DELETE CASCADE ON UPDATE CASCADE
154 )
155 TABLESPACE pg_default;
156
157 ALTER TABLE IF EXISTS public.orders
158     OWNER TO postgres;
159
160
161
162 -- PART 2: ADD MISSING CONSTRAINTS (FOREIGN KEYS)
163
164 ALTER TABLE customer
165 ADD CONSTRAINT fk_c_nationkey
166 FOREIGN KEY (c_nationkey) REFERENCES nation(n_nationkey)
167 ON DELETE CASCADE ON UPDATE CASCADE
168
169 ALTER TABLE lineitem
170 ADD CONSTRAINT fk_l_orderkey
171 FOREIGN KEY (l_orderkey) REFERENCES orders(o_orderkey)
172 ON DELETE CASCADE ON UPDATE CASCADE
173
174 ALTER TABLE lineitem
175 ADD CONSTRAINT fk_l_partsupp
176 FOREIGN KEY (l_partkey, l_suppkey) REFERENCES partsupp(ps_partkey, ps_suppkey)
177 ON DELETE CASCADE ON UPDATE CASCADE
178
179
180 ALTER TABLE nation
181 ADD CONSTRAINT fk_n_regionkey
182 FOREIGN KEY (n_regionkey) REFERENCES region(r_regionkey)
183 ON DELETE CASCADE ON UPDATE CASCADE
184

```

```

185
186
187 -- PART 3: ADD MISSING CONSTRAINTS (CHECKS)
188
189 -- Part table
190 ALTER TABLE part
191 ADD CONSTRAINT part_partkey_check CHECK (p_partkey >= 0);
192
193 -- Supplier table
194 ALTER TABLE supplier
195 ADD CONSTRAINT supplier_suppkey_check CHECK (s_suppkey >= 0);
196
197 -- Customer table
198 ALTER TABLE customer
199 ADD CONSTRAINT customer_custkey_check CHECK (c_custkey >= 0);
200
201 -- Partsupp table
202 ALTER TABLE partsupp
203 ADD CONSTRAINT partsupp_partkey_check CHECK (ps_partkey >= 0);
204
205 -- Region table
206 ALTER TABLE region
207 ADD CONSTRAINT region_regionkey_check CHECK (r_regionkey >= 0);
208
209 -- Nation table
210 ALTER TABLE nation
211 ADD CONSTRAINT nation_nationkey_check CHECK (n_nationkey >= 0);
212
213 -- Part table
214 ALTER TABLE part
215 ADD CONSTRAINT part_size_check CHECK (p_size >= 0),
216 ADD CONSTRAINT part_retailprice_check CHECK (p_retailprice >= 0);
217
218 -- Partsupp table
219 ALTER TABLE partsupp
220 ADD CONSTRAINT partsupp_availqty_check CHECK (ps_availqty >= 0),
221 ADD CONSTRAINT partsupp_supplycost_check CHECK (ps_supplycost >= 0);
222
223 -- Orders table
224 ALTER TABLE orders
225 ADD CONSTRAINT orders_totalprice_check CHECK (o_totalprice >= 0);
226
227 -- Lineitem table
228 ALTER TABLE lineitem
229 ADD CONSTRAINT lineitem_quantity_check CHECK (l_quantity >= 0),
230 ADD CONSTRAINT lineitem_extendedprice_check CHECK (l_extendedprice >= 0),
231 ADD CONSTRAINT lineitem_tax_check CHECK (l_tax >= 0);

```

```
232
233  -- Lineitem table
234  ALTER TABLE lineitem
235  ADD CONSTRAINT lineitem_discount_range_check CHECK (l_discount >= 0.00 AND
↪   l_discount <= 1.00),
236  ADD CONSTRAINT lineitem_shipdate_receiptdate_check CHECK (l_shipdate <=
↪   l_receiptdate);
```
